Лабораторная работа №2

Основные модели

Виктория Симонова

Содержание

1 Цель работы **5**

2 Задание **6**

3 Выполнение лабораторной работы **7**

3.1 Sir 7

3.2 Sir_params 21

3.3 Модель Лотки–Вольтерры 24

3.4 lv_params 39

4 Выводы **42**

Список литературы **43**

Список иллюстраций

3.1 рабочее пространство . . . 7
3.2 пакеты_sir . . . 7
3.3 графики двух моделей . . . 7
3.4 clean_sir . . . 8
3.5 jupyter_sir . . . 8
3.6 sir_effective_R . . . 19
3.7 sir_infected . . . 19
3.8 sir_log_scale . . . 19
3.9 sir_main . . . 20
3.10 sir_panel . . . 20
3.11 sir_percentages . . . 21
3.12 sir_phase_portrait . . . 21
3.13 comparison_beta . . . 24
3.14 графики двух моделей . . . 24
3.15 clean_lvr . . . 25
3.16 jupyter_lv . . . 25
3.17 lv_derivatives . . . 37
3.18 lv_panel . . . 37
3.19 lv_dynamics . . . 38
3.20 lv_phase_portrait . . . 38
3.21 lv_relative_changes . . . 39
3.22 lv_spectrum . . . 39
3.23 params_lv . . . 40
3.24 params_lv . . . 41
3.25 lit_code . . . 41

Список таблиц

1 Цель работы

Приобретение практических навыков построения имитационных моделей на языке Julia Модель SIR и Модель Лотки–Вольтерры , освоение инструментов автоматизации подготовки отчетов (DrWatson, Literate, Quarto) .

2 Задание

— Создать рабочий каталог для кода. — Установить необходимые пакеты. — Выполнить предложенный код. — Преобразовать код в литературный стиль. — Сгенерировать из литературного кода: — чистый код; — jupyter notebook; — документацию в формате Quarto. — Выполнить код из jupyter notebook. — Интегрировать документацию в формате Quarto в отчёт. — Добавить в код в литературном стиле вычисление для набора параметров. — Сгенерировать из литературного кода с параметрами: — чистый код; — jupyter notebook; — документацию в формате Quarto. — Выполнить код из jupyter notebook с параметрами. — Интегрировать документацию с параметрами в формате Quarto в отчёт

3 Выполнение лабораторной работы

3.1 Sir

Подготовка рабочего пространства (рис. 3.1).

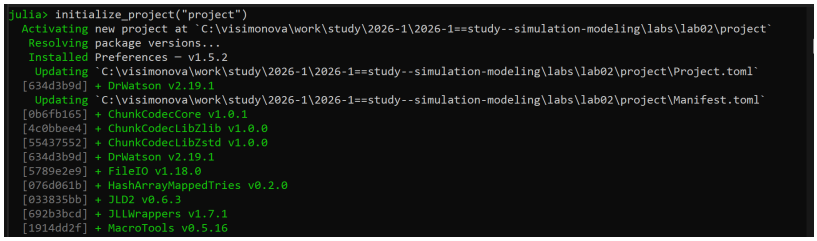


Рисунок 3.1: рабочее пространство

Ставлю необходимые Пакеты (рис. 3.2).

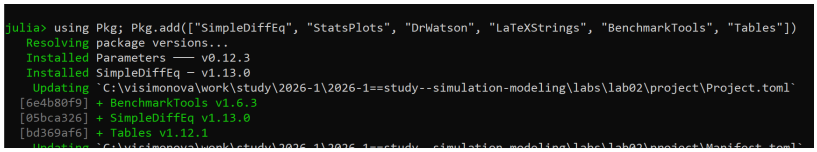


Рисунок 3.2: пакеты_sir

Выполнение предложенного кода обоих моделей, получились графики (рис. 3.3).

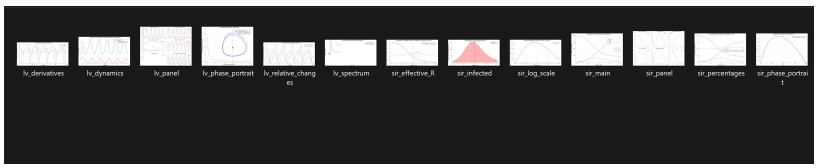
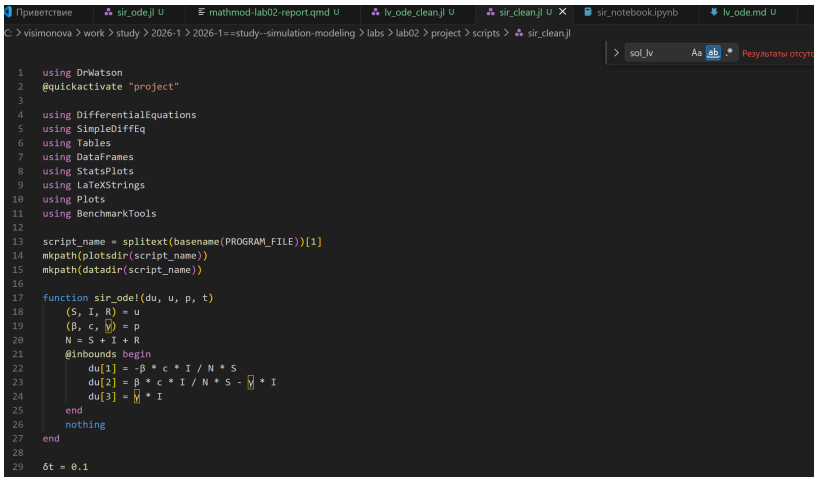


Рисунок 3.3: графики двух моделей

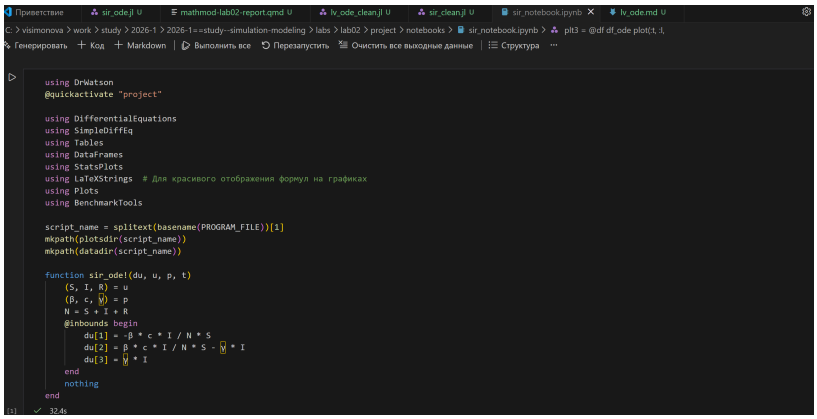
Генерация чистого кода (рис. 3.4).



```julia
using DrWatson
@quickactivate "project"

using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p
    N = S + I + R
    @inbounds begin
        du[1] = -β * c * I / N * S
        du[2] = β * c * I / N * S - γ * I
        du[3] = γ * I
    end
    nothing
end

δt = 0.1
```

Рисунок 3.4: clean_sir

Генерация jupyter notebook (рис. 3.5).



```julia
using DrWatson
@quickactivate "project"

using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings  # Для красивого отображения формул на графиках
using Plots
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p
    N = S + I + R
    @inbounds begin
        du[1] = -β * c * I / N * S
        du[2] = β * c * I / N * S - γ * I
        du[3] = γ * I
    end
    nothing
end
```

Рисунок 3.5: jupyter_sir

3.1.1 Реализация модели

```julia
using DrWatson
@quickactivate "project"
```

```julia
using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings  # □□□ □□□□□□□□□ □□□□□□□□□□□ □□□□□□ □□ □□□□□□□□
using Plots
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (□, c, □) = p
    N = S + I + R
    @inbounds begin
        du[1] = -□ * c * I / N * S
        du[2] = □ * c * I / N * S - □ * I
        du[3] = □ * I
    end
    nothing
end

# ## □□□□□□□□□□□ □□□□□□□□□□□ □ □□□□□□
# □□□□□□□□□ □□□□□□□ □□□□□ □□□□□□□□□ $c$ □□ □□□□□□□□ □□□□□□□□.
params_dict = Dict(
```

```
    :□ => [0.05],              # □□□□□□□□□□□ □□□□□□□□□
    :c => [5.0, 10.0, 15.0], # □□□□□□□□ □□□□□□□□□
    :□ => [0.25]               # □□□□□□□□ □□□□□□□□□□□□□
)

dicts = dict_list(params_dict)

# □□□□□ □□□□□□□□□
□t, tmax = 0.1, 60.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0]

# □□□□□□□□□□ □□□ □□□□□□□□ □□□□□□□□□ □□□□□□□□□□□ (□□□□□□□□□□ □□□ Literate)
global sol_ode = nothing
global prob_ode = nothing # □□□□□□□□ □□□□
global df_ode = nothing
global p = []
global R0 = 0.0

# ### □□□□ □□□□□□□□□□□□□
for (i, d) in enumerate(dicts)
    local_p = [d[:□], d[:c], d[:□]]
    local_R0 = (local_p[2] * local_p[1]) / local_p[3]

    local_prob = ODEProblem(sir_ode!, u0, tspan, local_p) # □□□□□□□ □
    local_sol = solve(local_prob, dt = □t)

    ## □□□□□□□□□ □□□□□□□□□ □□□□□□ □□□ □□□□□□□□□□ □□□□□□□ □□□□
```

```
    if i == length(dicts)
        global sol_ode = local_sol
        global prob_ode = local_prob # □□□□□□□□□□□ □□□□□□□ □□□□□□!
        global p = local_p
        global R0 = local_R0
        global df_ode = DataFrame(Tables.table(sol_ode'))
        rename!(df_ode, ["S", "I", "R"])
        df_ode[!, :t] = sol_ode.t
        df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R
    end

    println("□□□□□ $i: c = $(d[:c]), R₀ = $(round(local_R0, digits=2
end

# ### □□□□ □□□□□□□□□□□□□
for (i, d) in enumerate(dicts)
    local_p = [d[:□], d[:c], d[:□]]
    local_R0 = (local_p[2] * local_p[1]) / local_p[3]

    prob_ode = ODEProblem(sir_ode!, u0, tspan, local_p)
    local_sol = solve(prob_ode, dt = □t)

    ## □□□□□□□□□ □□□□□□□□□ □□□□□□ □□□ □□□□□□□□□□ □□□□□□□ □□□□
    if i == length(dicts)
        global sol_ode = local_sol
        global p = local_p
        global R0 = local_R0
        global df_ode = DataFrame(Tables.table(sol_ode'))
```

```
        rename!(df_ode, ["S", "I", "R"])
        df_ode[!, :t] = sol_ode.t
        df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R
    end

    println("□□□□□ $i: c = $(d[:c]), R₀ = $(round(local_R0, digits=2
end


# □□□□□ □□□□□□□□□□ □□□□□□
println("□□□□□□□□□ □□□□□□ SIR:")
println("□ (□□□□□□□□□□□ □□□□□□□□□) = ", p[1])
println("c (□□□□□□□ □□□□□ □□□□□□□□□) = ", p[2])
println("□ (□□□□□□□□ □□□□□□□□□□□□□) = ", p[3])
println("R₀ = c * □ / □ = ", round(R0, digits=3))
println("□□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□□□ = ", round(1/p[3], digits=2), "
println("□□□□□□□□□ □□□□□□□□: S₀ = ", u0[1], ", I₀ = ", u0[2], ", R₀ = ",

# 1. □□□□□□□□ □□□□□□: □□□□□□□□□ □□□□□ □□□□□ □□□□□□
plt1 = @df df_ode plot(:t,
    [:S :I :R],
    label=[L"S(t)" L"I(t)" L"R(t)"],
    xlabel="□□□□□□, □□□□",
    ylabel="□□□□□□□□□□□ □□□□□□",
    title="□□□□□□ SIR: □□□□□□□□□ □□□□□□□□□",
    linewidth=2,
    legend=:right,
    grid=true,
```

```
    size=(800, 500))

# □□□□□□□□□□ □□□□□□□□□□ □ □□□□□□□□□□□□
annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
    text("□□□□□□□□□□: □ = $(p[1]) c = $(p[2]) □ = $(p[3]) R₀ = $(round(
    8, :left))

# □□□□□□ □□□□□□ □□□□□□□□□□□□□□ (I)
plt2 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□",
    title="□□□□□□□□ □□□□□ □□□□□□□□□□",
    color=:red,
    linewidth=2,
    fill=(0, 0.3, :red),
    grid=true,
    size=(800, 400))

# □□□□□□□ □□□□ □□□□□□□□
peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false
annotate!(plt2, peak_time, peak_value * 1.05,
    text("□□□: $(round(peak_value, digits=1)) □□ $(round(peak_time, d
    8, :top))
```

```
# □□□□□□ □ □□□□□□□□□□□□□□□ □□□□□□□□ (□□□ □□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□)
plt3 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□ (□□□. □□□□□□□)",
    title="□□□□□□□□□□□□□□□□ □□□□ (□□□. □□□□□)",
    yscale=:log10,
    color=:red,
    linewidth=2,
    grid=true,
    size=(800, 400))

# □□□□□□ □□□□□ □□□□□□□□□ (□ □□□□□□□□□)
plt4 = @df df_ode plot(:t,
    [:S :I :R] ./ df_ode.N .* 100,
    label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
    xlabel="□□□□□, □□□",
    ylabel="□□□□ □□□□□□□□□, %",
    title="□□□□□□□□ □□□□□□□□ (□ □□□□□□□□□)",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))

# □□□□□□□□□□□□□□ □□□□□ □□□ □□□□□□ □□□□□□□□□□□□ □□□□□□□□□□
if R0 > 1
    herd_immunity_threshold = (1 - 1/R0) * 100
    hline!(plt4, [herd_immunity_threshold], color=:purple, linestyl
```

```
        label="□□□□□ □□□□□□□□□□□□□ □□□□□□□□□□ ($(round(herd_immunity_thres
        linewidth=1.5)
end

# □□□□□□□ □□□□□□□ (I vs S)
plt5 = plot(df_ode.S, df_ode.I,
    label="□□□□□□□ □□□□□□□□□□",
    xlabel=L"S(t)",
    ylabel=L"I(t)",
    title="□□□□□□□ □□□□□□□ SIR □□□□□□",
    color=:blue,
    linewidth=2,
    grid=true,
    size=(800, 500),
    legend=:topright)

# □□□□□□□□□□ □□□□□□□ □□□□□□□□□□□
for i in 1:50:length(df_ode.S)-1
    plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i], df_ode.
        arrow=:closed, color=:blue, alpha=0.5, label=false)
end

# □□□□□□ Rₑ - □□□□□□□□□□□□□ □□□□□□□□□□□□□□□□ □□□□□
df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N
plt6 = @df df_ode plot(:t, :Re,
    label=L"R_e(t)",
    xlabel="□□□□□, □□□",
    ylabel=L"R_e",
```

```
    title="□□□□□□□□ □□□□□□□□□□□□ □□□□□□□□□□□□□□□ □□□□□□",
    color=:green,
    linewidth=2,
    grid=true,
    size=(800, 400))

# □□□□□□□□□□□□□□□ □□□□□□ □□ □□□□□□□ 1
hline!(plt6, [1.0], color=:red, linestyle=:dash, label="□□□□□ □□□□□□□□□

# □□□□□□□□ □□□□□□□□□, □□□□□□ Rₑ □□□□□□□□□□□ < 1
cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
    cross_time = df_ode.t[cross_idx]
    vline!(plt6, [cross_time], color=:black, linestyle=:dash, label=
    annotate!(plt6, cross_time, 1.2,
        text("Rₑ<1 □ $(round(cross_time, digits=1)) □□□", 8, :left))
end

# □□□□□□□□□□□ □□□□□□□ □□□□□ □□□□□□□ □ □□□□□□ □□□□□□□
plt7 = plot(layout=(2, 3), size=(1200, 800))

# □□□□□□□□ □□□□
plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1, linewidth
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidth
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3, linewidth

# □□□□□□□ □□□□
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidth
```

```
    yscale=:log10, title="□□□. □□□□□□□")
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4, linewidth=
    title="□□□□□□□ □□□□□□□", xlabel=L"S", ylabel=L"I")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, lin
    title=L"R_e(t)", hline=[1.0], linestyle=:dash, linecolor=:red)

# □□□□□□□□□□ □□□□□□□□

savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))

# □□□□□□□□ □□□ □□□□□□ □□□□□□□□□□□□□□□□□
println(" □□□□□□□□ □□□□□□□:")
@benchmark solve(prob_ode, dt = □t)

# □□□□□□□□□□□□□□ □□□□□□
println(" === □□□□□□ □□□□□□□□□□□ ===")
println("□□□□□ □□□□□□□□□□□ □□□□□□□□□ (□□□□□□□□): N = ", round(df_ode.N[1],
println("□□□□□□□ □□□□□ □□□□□□□□□□: I_max = ", round(peak_value, digits=1
println("□□□□□ □□□□□□□□□□ □□□□: t_peak = ", round(peak_time, digits=1),
println("□□□□□□□□ □□□□□ □□□□□□□□□□□□: R(∞) = ", round(df_ode.R[end], digi
println("□□□□ □□□□□□□□□□□□: ", round(df_ode.R[end]/df_ode.N[1]*100, dig
```

```
if R0 > 1
    println(" □□□□□□□□□□□□□ □□□□□□:")
    println("  - □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: ", round((1-1/R0)*100, di
    println("  - □□□□□□□□□□□□□ □□□ □□□ S/N = 1/R₀ = ", round(1/R0, digit
end

# # □□□□□□ □□□□□□ SIR □ □□□□□□□ □□□□□□□□□□□
# □□□□□□□□□ □□□□□□□ □□□□□□□□□□□ □□□□□□□□□ (□) □□ □□□ □□□□□□□□.

# ## □□□□□□□□□□□ □□□□□□ □□□□□□□□□□
beta_values = [0.03, 0.05, 0.08] # □□□□□□ □□□□□□□□ □
plt_comparison = plot(title="□□□□□□□ □ □□ □□□□□□□□□□ □□□□□□□□□□□□□□", xlabe

for b in beta_values
    local p_new = [b, 10.0, 0.25] # □□□□□□□□□ □
    local prob = ODEProblem(sir_ode!, u0, tspan, p_new)
    local sol = solve(prob, dt = □t)

    ## □□□□□□□□□ □□□□□□ □□ □□□□□ □□□□□□
    plot!(plt_comparison, sol.t, [u[2] for u in sol.u], label="□ = S
end

# ## □□□□□□□□□□□ □ □□□□□□□□□□ □□□□□□□□□□
# □□□□□□□□□ □□□□□□□□□□□□□ □□□□□□
savefig(plt_comparison, plotsdir(script_name, "comparison_beta.png"
plt_comparison
```

Результаты (рис. 3.6).

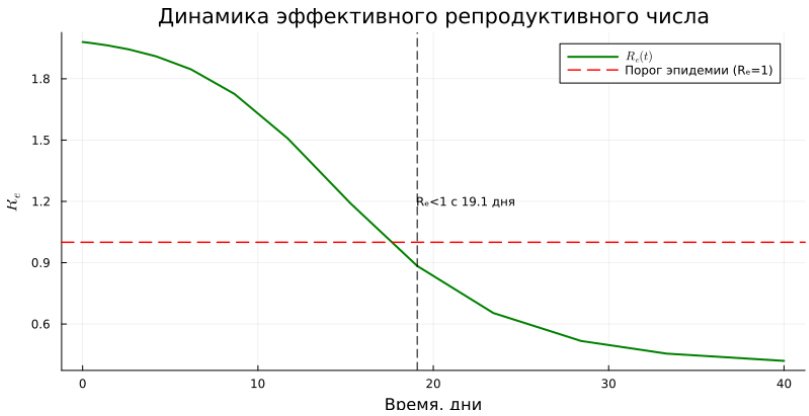


Рисунок 3.6: sir_effective_R

Результаты (рис. 3.7).

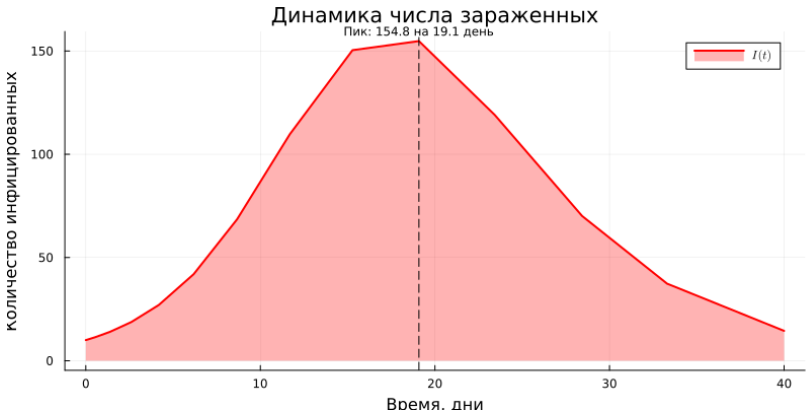


Рисунок 3.7: sir_infected

Результаты (рис. 3.8).

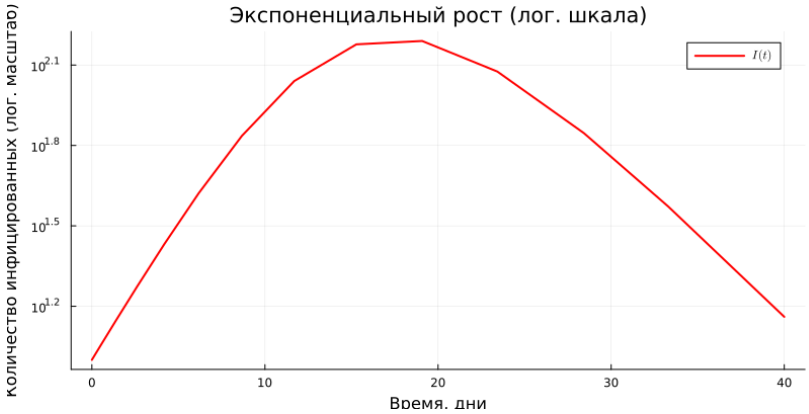


Рисунок 3.8: sir_log_scale

Результаты (рис. 3.9).

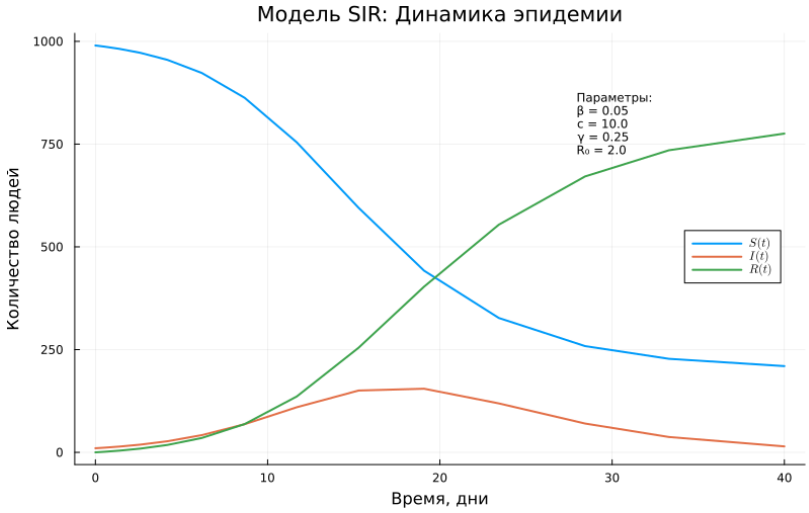


Рисунок 3.9: sir_main

Результаты (рис. 3.10).

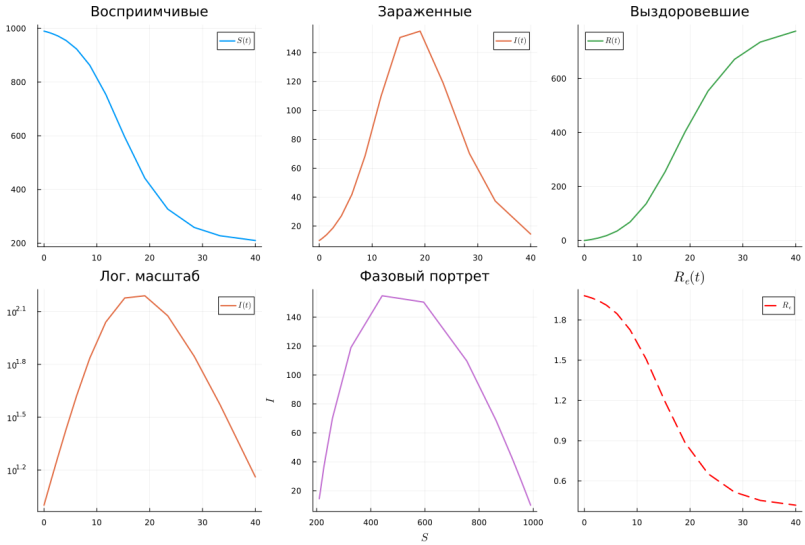


Рисунок 3.10: sir_panel

Результаты (рис. 3.11).

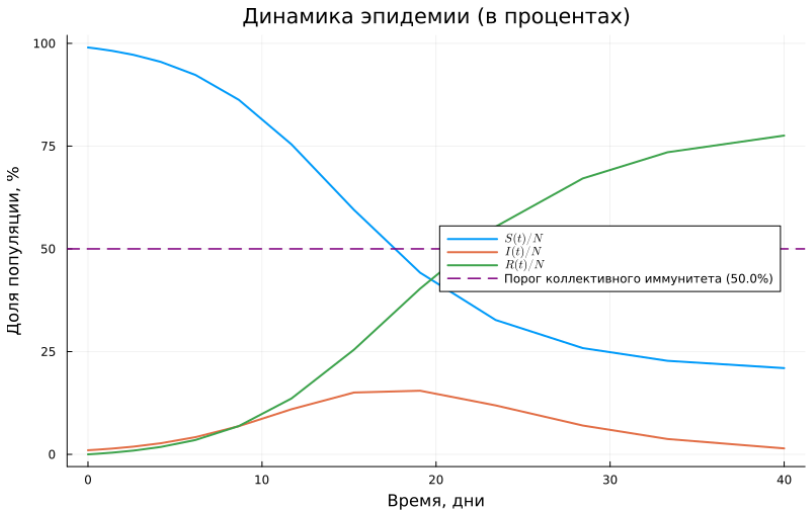


Рисунок 3.11: sir_percentages

Результаты (рис. 3.12).

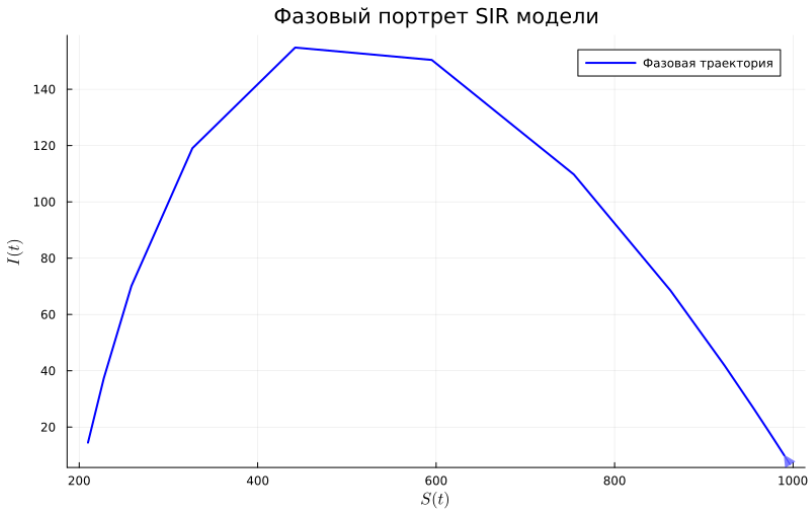


Рисунок 3.12: sir_phase_portrait

3.2 Sir_params

заменила

```
# □□□□□□□□□ □□□□□□
□t = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0]  # S, I, R
p = [0.05, 10.0, 0.25]   # □, c, □

# □□□□□□ □□□□□□□□ □□□□□□□□□□□□□□□ □□□□□
R0 = (p[2] * p[1]) / p[3]  # R₀ = (c * □) / □

# □□□□□□□□ □ □□□□□□□ □□□□□□
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = □t)
```

на

```
# □□□□□□□□□□□ □□□□□□ □□□□□□□□□□ □□□ □□□□□□□□
params_dict = Dict(
    :□ => [0.05],           # □□□□□□□□□□□ □□□□□□□□□
    :c => [5.0, 10.0, 15.0], # □□□□□□ □□□□□□□□ □□□□□□□□ □□□□□ □□□□□□□□□□
    :□ => [0.25]            # □□□□□□□□ □□□□□□□□□□□□□
)

dicts = dict_list(params_dict)

# □□□□□ □□□□□□□□□ □□□□□□□
□t = 0.1
tmax = 60.0
tspan = (0.0, tmax)
```

```
u0 = [990.0, 10.0, 0.0]

# □□□□ □□ □□□□ □□□□□□□□□□ □□□□□□□□□□
for (i, d) in enumerate(dicts)
    p = [d[:□], d[:c], d[:□]]
    R0 = (p[2] * p[1]) / p[3]

    # □□□□□□□
    prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
    sol_ode = solve(prob_ode, dt = □t)

    # □□□□□□□□ DataFrame
    df_temp = DataFrame(Tables.table(sol_ode'))
    rename!(df_temp, ["S", "I", "R"])
    df_temp[!, :t] = sol_ode.t

    # □□□□□□□□□□ □□□□□□ □□□ □□□□□□□ □□□□□□ (□ □□□□□□□□□□ □□□□□□)
    filename = "sir_data_c$(d[:c]).csv"
    safesave(datadir(script_name, filename), df_temp)

    println("□□□□□□□□□ □□□□□ $i: c = $(d[:c]), R₀ = $(round(R0, digits=

    # □□□□□ □□□□□□□□□ sol_ode □□□ df_temp □□□ □□□□□□□□□□ □□□□□□□□ □□□□□□ □□□□□
    # □□□ □□□□□□□ □□□□□□□□□ □□□□□□ □□□ □□□□□□□□□□□□□ □ □□□□□ □□□□□ □□□□
    global sol_ode = sol_ode
    global df_ode = df_temp
    global p = p
    global R0 = R0
```

```
end
```

Показываю, как меняется эпидемия при разных сценариях (рис. 3.13).

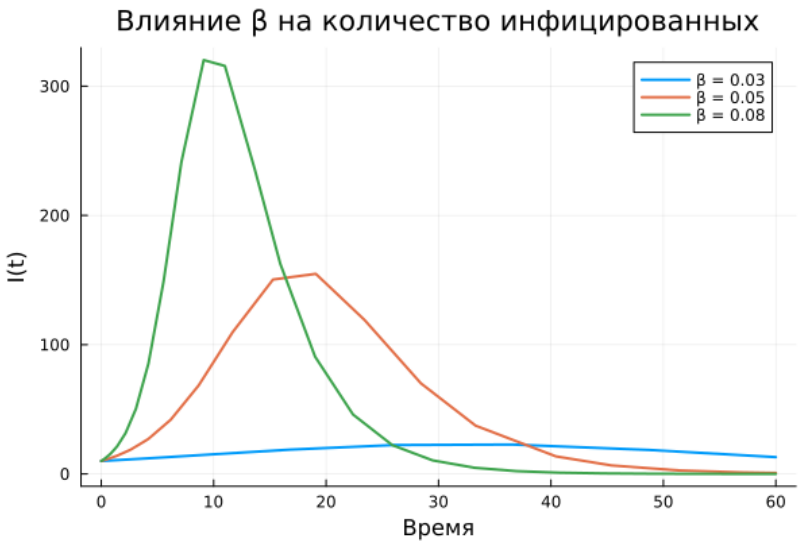


Рисунок 3.13: comparison_beta

3.3 Модель Лотки–Вольтерры

Выполнение предложенного кода обоих моделей, получились графики (рис. 3.14).

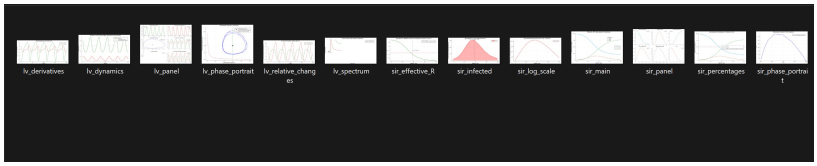
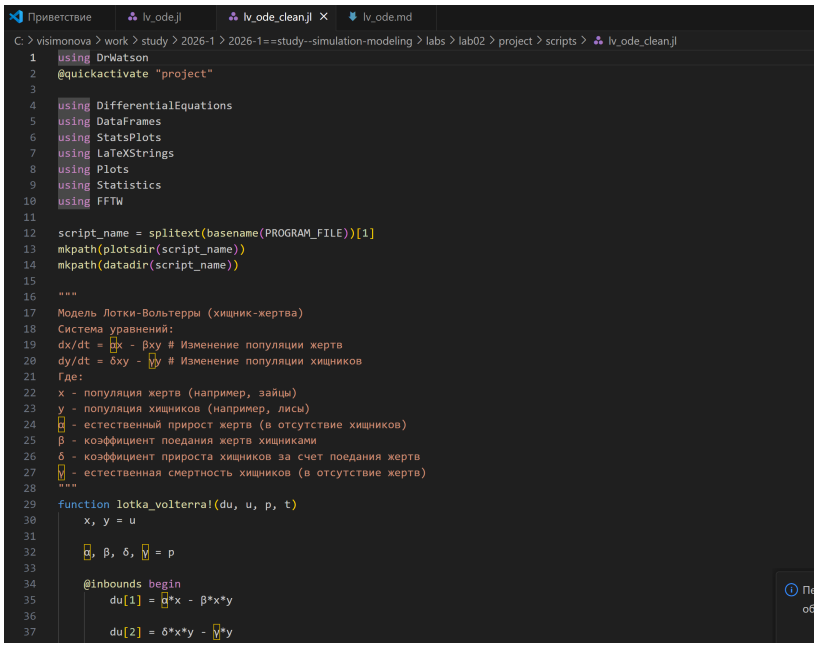


Рисунок 3.14: графики двух моделей

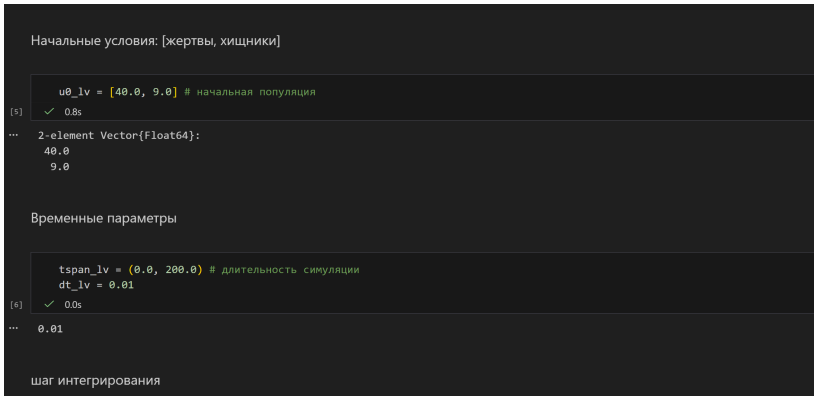
Генерация чистого кода (рис. 3.15).



```julia
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

"""
Модель Лотки-Вольтерры (хищник-жертва)
Система уравнений:
dx/dt = αx - βxy # Изменение популяции жертв
dy/dt = δxy - γy # Изменение популяции хищников
Где:
x - популяция жертв (например, зайцы)
y - популяция хищников (например, лисы)
α - естественный прирост жертв (в отсутствие хищников)
β - коэффициент поедания жертв хищниками
δ - коэффициент прироста хищников за счет поедания жертв
γ - естественная смертность хищников (в отсутствие жертв)
"""
function lotka_volterra!(du, u, p, t)
    x, y = u

    α, β, δ, γ = p

    @inbounds begin
        du[1] = α*x - β*x*y

        du[2] = δ*x*y - γ*y
```

Рисунок 3.15: clean_lvr

Генерация jupyter notebook (рис. 3.16).



Начальные условия: [жертвы, хищники]

```julia
u0_lv = [40.0, 9.0] # начальная популяция
```

[5] ✓ 0.8s

```
2-element Vector{Float64}:
 40.0
  9.0
```

Временные параметры

```julia
tspan_lv = (0.0, 200.0) # длительность симуляции
dt_lv = 0.01
```

[6] ✓ 0.0s

```
0.01
```

шаг интегрирования

Рисунок 3.16: jupyter_lv

3.3.1 Реализация модели

```julia
using DrWatson
@quickactivate "project"
```

```julia
using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

# □□□□□□□□ □□□□□□ □□□□□-□□□□□□□□□
"""
□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)
□□□□□□□ □□□□□□□□□:
dx/dt = □x - □xy # □□□□□□□□□ □□□□□□□□□ □□□□□
dy/dt = □xy - □y # □□□□□□□□□ □□□□□□□□□ □□□□□□□□
□□□:
x - □□□□□□□□□ □□□□□ (□□□□□□□□, □□□□□)
y - □□□□□□□□□ □□□□□□□□ (□□□□□□□□, □□□□)
□ - □□□□□□□□□□□□ □□□□□□□ □□□□□ (□ □□□□□□□□□□ □□□□□□□□)
□ - □□□□□□□□□□□ □□□□□□□□ □□□□□ □□□□□□□□□
□ - □□□□□□□□□□□ □□□□□□□□ □□□□□□□□ □□ □□□□ □□□□□□□□ □□□□□
□ - □□□□□□□□□□□ □□□□□□□□□ □□□□□□□□ (□ □□□□□□□□□□ □□□□□)
"""
# # □□□□□□□□□□ □□□□□□
# □□□□□□□□□ □□□□□□□ □□□□□□□□□□□□□□□□ □□□□□□□□□.
```

```
function lotka_volterra!(du, u, p, t)
    x, y = u
    □, □, □, □ = p
    @inbounds begin
        du[1] = □*x - □*x*y
        du[2] = □*x*y - □*y
    end
    nothing
end

# □□□□□□□□□ □□□□□□ □ □□□□□□□□□ □□□□□□□
# □□□□□□□□□□□□□ □□□□□□□□□ □□ □□□□□□□□□□
p_lv = [0.1, 0.2, 0.3, 0.4]
# □: □□□□□□□□□□ □□□□□□□□□

# □□□□□□□□□ □□□□□□□: [□□□□□□, □□□□□□□]
u0_lv = [40.0, 9.0] # □□□□□□□□□ □□□□□□□□□□

# □□□□□□□□□ □□□□□□□□□
tspan_lv = (0.0, 200.0) # □□□□□□□□□□□□ □□□□□□□□□
dt_lv = 0.01
# □□□ □□□□□□□□□□□□□□

prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)
sol_lv = solve(prob_lv, Tsit5(), dt = dt_lv, adaptive = true)
## □□□ □□□□□□□□□ solve □□□□□□ □□□□ □ □□□□□ □□□□□ □□□ □□□□□□□□

# □□□□□□□□□□ □□□□□□
```

```
df_lv = DataFrame()
df_lv[!, :t] = sol_lv.t
df_lv[!, :prey] = [u[1] for u in sol_lv.u] # □□□□□□
df_lv[!, :predator] = [u[2] for u in sol_lv.u] # □□□□□□□

# □□□□□□□ □□□□□□□□□□□ □□□ □□□□□□□
df_lv[!, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey
df_lv[!, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .

# □□□□□ □□□□□□□□□□ □ □□□□□□
println("="^60)
println("□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)")
println("="^60)
println(" □□□□□□□□□ □□□□□□:")
println("□ (□□□□□□□□ □□□□□□□□□□ □□□□□) = ", p_lv[1])
println("□ (□□□□□□□□ □□□□□□□□ □□□□□) = ", p_lv[2])
println("□ (□□□□□□□□□□ □□□□□□□□□) = ", p_lv[3])
println("□ (□□□□□□□□□□ □□□□□□□□) = ", p_lv[4])
println(" □□□□□□□□□ □□□□□□□:")
println("□□□□□□ (x0) = ", u0_lv[1])
println("□□□□□□□ (y0) = ", u0_lv[2])

# □□□□□□□□□□□□ □□□□□ (□□□□□□□ □□□□□□□□)
x_star = p_lv[4] / p_lv[3] # □□□□□□□□□□□□ □□□□□ □□□ □□□□□
y_star = p_lv[1] / p_lv[2] # □□□□□□□□□□□□ □□□□□ □□□ □□□□□□□□
println(" □□□□□□□□□□□□ □□□□□ (□□□□□□□□□ □□□□□□□□□□):")
println("x* = □/□ = ", round(x_star, digits=3))
println("y* = □/□ = ", round(y_star, digits=3))
```

```
# □□□□□□□□□□ □□□□□□□□
# □□□□□□ 1: □□□□□□□□ □□□□□□□□□ □□ □□□□□□□
plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
    label=[L"□□□□□□ (x)" L"□□□□□□□ (y)"],
    xlabel="□□□□□",
    ylabel="□□□□□□□□□",
    title="□□□□□□ □□□□□-□□□□□□□□□: □□□□□□□□ □□□□□□□□□",
    linewidth=2,
    legend=:topright,
    grid=true,
    size=(900, 500),
    color=[:green :red])

# □□□□□□□□□□ □□□□□□□□□□□□ □□□□□□□
hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5, la
hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5, labe

# □□□□□□ 2: □□□□□□□ □□□□□□□ (□□□□□□□ vs □□□□□□)
plt2 = plot(df_lv.prey, df_lv.predator,
    label="□□□□□□□ □□□□□□□□□□",
    xlabel="□□□□□□□□□ □□□□□ (x)",
    ylabel="□□□□□□□□□ □□□□□□□□ (y)",
    title="□□□□□□□ □□□□□□□ □□□□□□□",
    color=:blue,
    linewidth=1.5,
    grid=true,
    size=(800, 600),
    legend=:topright)
```

```
# □□□□□□□□□□ □□□□□□□ □□□□□□□□□□□ □□ □□□□□□□ □□□□□□□□
step = 50 # □□□ □□□ □□□□□□□□□□□ □□□□□□□
for i in 1:step:length(df_lv.prey)-step
    plot!(plt2, [df_lv.prey[i], df_lv.prey[i+step]],
          [df_lv.predator[i], df_lv.predator[i+step]],
          arrow=:closed, color=:blue, alpha=0.3, label=false)
end

# □□□□□□□□□□ □□□□□□□□□□□□ □□□□□
scatter!(plt2, [x_star], [y_star],
    color=:black, markersize=8, label="□□□□□□□□□□□□ □□□□□ (x*, y*)")

# □□□□□□□□ (□□□□□□□□ □□□□□)
x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
y_nullcline = p_lv[1] ./ (p_lv[2] .* x_range) # y-□□□□□□□□ (dy/dt = 0

plot!(plt2, x_range, y_nullcline,
    color=:red, linestyle=:dash, linewidth=1.5, label="□□□□□□□□ □□□□□□

y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
x_nullcline = p_lv[4] ./ (p_lv[3] .* ones(length(y_range))) # x-□□□□

plot!(plt2, x_nullcline, y_range,
    color=:green, linestyle=:dash, linewidth=1.5, label="□□□□□□□□ □□□□

# □□□□□□ 3: □□□□□□□□□□□ (□□□□□□□□ □□□□□□□□□)
plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
    label=[L"dx/dt" L"dy/dt"],
```

```
    xlabel="□□□□□",
    ylabel="□□□□□□□□ □□□□□□□□□",
    title="□□□□□□□□□□□ □□□□□□□□□",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),
    color=[:green :red])

hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3, label=

# □□□□□□ 4: □□□□□□□□□□□□□ □□□□□□□□□ (□ %)
df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.predat

plt4 = plot(df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_chan
    label=[L"dx/dt / x (\%)" L"dy/dt / y (\%)"],
    xlabel="□□□□□",
    ylabel="□□□□□□□□□□□□□ □□□□□□□□□, %",
    title="□□□□□□□□□□□□□ □□□□□ □□□□□",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),
    color=[:green :red])

# □□□□□□ 5: □□□□□□□□□□□□ □□□□□□ (□□□□□□□ □□□□□□□□□□□□□□□ □□□□□)
function compute_fft(signal, dt)
```

```
    n = length(signal)
    # □□□□□□□□□□ rfft □□□ □□□□□□□□□□□□ □□□□□□□□ (□□□□□□□□□□ □□□□□□ □□□□□□□□□□□□□□
    spectrum = abs.(rfft(signal))
    # □□□□□□□□□□□□□□□□ □□□□□□□ □□□ rfft
    freq = rfftfreq(n, 1/dt)
    return freq, spectrum
end

# □□□□□□□□□□ □□□□□□□□ □□□□□□□□□
freq_prey, spectrum_prey = compute_fft(df_lv.prey .- mean(df_lv.prey
freq_predator, spectrum_predator = compute_fft(df_lv.predator .- mea

plt5 = plot(freq_prey, [spectrum_prey spectrum_predator],
    label=[L"□□□□□□ (x)" L"□□□□□□□ (y)"],
    xlabel="□□□□□□□",
    ylabel="□□□□□□□□□",
    title="□□□□□□□□□□□□ □□□□□□ (□□□□□)",
    linewidth=1.5,
    xscale=:log10,
    yscale=:log10,
    legend=:topright,
    grid=true,
    size=(800, 400),
    color=[:green :red])

# □□□□□□□□□□ □□□□□□□□□□□□ □□□□□□
if length(spectrum_prey) > 0
    idx_prey = argmax(spectrum_prey[2:end]) + 1 # □□□□□□□□□□ □□□□□□□ □□□
```

```
    dominant_freq_prey = freq_prey[idx_prey]
    period_prey = 1/dominant_freq_prey
    println(" □□□□□□□□□□□□ □□□□□□□ □□□□□□□□□ □□□□□: ", round(dominant_freq_
    println("□□□□□□ □□□□□□□□□ □□□□□: ", round(period_prey, digits=2), "
end

# □□□□□□ 6: □□□□□□□□□□ □□□□□□ □□□□ □□□□□□□□
plt6 = plot(layout=(3, 2), size=(1200, 900))

plot!(plt6[1], df_lv.t, df_lv.prey, label=L"x(t)", color=:green, li
plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red, 
plot!(plt6[3], df_lv.prey, df_lv.predator, label=false, color=:blue
scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5, la
plot!(plt6[4], df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt], label=
plot!(plt6[5], freq_prey, spectrum_prey, label=L"x", color=:green, 
plot!(plt6[6], df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_cl

# □□□□□□ □□□□□□□□□□□
println(" " * "="^60)
println("□□□□□□ □□□□□□□□□□□")
println("="^60)

println(" □□□□□□□□ □□□□□□□□□□:")
println("□□□□□□: min = ", round(minimum(df_lv.prey), digits=2),
        ", max = ", round(maximum(df_lv.prey), digits=2),
        ", mean = ", round(mean(df_lv.prey), digits=2))
println("□□□□□□□: min = ", round(minimum(df_lv.predator), digits=2),
        ", max = ", round(maximum(df_lv.predator), digits=2),
```

```
        ", mean = ", round(mean(df_lv.predator), digits=2))

# □□□□□□□□□□ □□□□□□ □□□□□□□□□ □□□ □□□□□□ □□□□□□□□□□
# □□□□□□ □□□□□□□□ □□□□□□□ □□□□□□ □□□, □□□□□□ □□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□□□□

# □□□□□□□ □□□□□ □□□□□□□ □□□□ □□□□□ (□□□□□□□ □□□□□□□□)
function find_first_peak(signal, time)
    for i in 2:length(signal)-1
        if signal[i] > signal[i-1] && signal[i] > signal[i+1]
            return time[i], signal[i]
        end
    end
    return NaN, NaN
end

peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey, df_lv
peak_time_predator, peak_value_predator = find_first_peak(df_lv.pre

if !isnan(peak_time_prey) && !isnan(peak_time_predator)
    phase_shift = peak_time_predator - peak_time_prey
    println(" □□□□□□ □□□□□□□□□:")
    println("□□□□□□ □□□ □□□□□: □□□□□ = ", round(peak_time_prey, digits=2
    println("□□□□□□ □□□ □□□□□□□□: □□□□□ = ", round(peak_time_predator, d
    println("□□□□□ □□□ (□□□□□□□ □□□□□□□): ", round(phase_shift, digits=2
end

# □□□□□□□□□□ □□□□□□□□
savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
```

```
savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))

println(" " * "="^60)
println("□□□□□□□□□□□□□ □□□□□□□□□ □□□□□□□! □□□□□□□ □□□□□□□□□ □ plots/$(script_
println("="^60)

# □□□□□□□ □□□ □□□□□□□ □□□□□□□ □□□□□□□□□□
function analyze_parameter_sensitivity(param_index, values, param_na
    println(" □□□□□□ □□□□□□□□□□□□□□□□ □ □□□□□□□□□: ", param_name)
    results = []
    for val in values
        p_test = copy(p_lv)
        p_test[param_index] = val
        prob_test = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_
        sol_test = solve(prob_test, dt = dt_lv)
        prey_end = sol_test.u[end][1]
        predator_end = sol_test.u[end][2]
        push!(results, (param=val, prey=prey_end, predator=predator
        println("$(param_name)=$(val): □□□□□□=$(round(prey_end, digit
    end
    return results
end

# □□□□□□ □□□□□□□□□□□□□□□□ □ □□□□□□□□□ □□□□□□□□□□
```

```
if false # □□□□□□□□□□ □ true □□□ □□□□□□□□□□ □□□□□□□
    println(" 1. □□□□□□□ □□□□□□□□ □□□□□□□□□□□ □□□□□ (□):")
    analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], "□")
    println(" 2. □□□□□□□ □□□□□□□□□□ □□□□□□□□ (□):")
    analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], "□")
end

println(" □□□□□□□□□□□□□ □□□□□□□□□ □□□□□□□!")

# # □□□□□□ □□□□□□ □□□ □□□□□□□□□ □□□□□□□□□□
# □□□□□□□□□ □□□□□□□ □□□□□□□□□□□□ □□□□□□□□□□□ □□□□□ (alpha) □□ □□□□□□□□.

alphas = [0.5, 1.0, 1.5]
p_base = [1.0, 0.1, 0.1, 0.4]

for a in alphas
    p_new = [a, p_base[2], p_base[3], p_base[4]]
    prob_new = ODEProblem(lotka_volterra!, u0, tspan, p_new)
    sol_new = solve(prob_new)
    ## □□□□□□ □□□□□□ □□□ □□□□□□□ □□□□□□□□□
    display(plot(sol_new, title="□□□□□□ □□□ alpha = $a"))
end
```

Результаты (рис. 3.17).

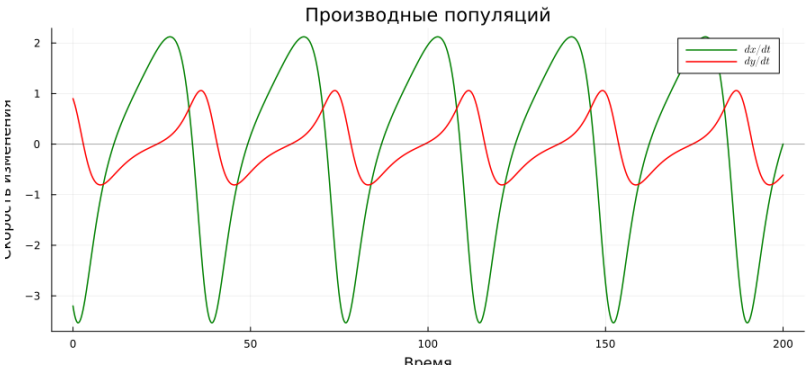


Рисунок 3.17: lv_derivatives

Результаты (рис. 3.18).

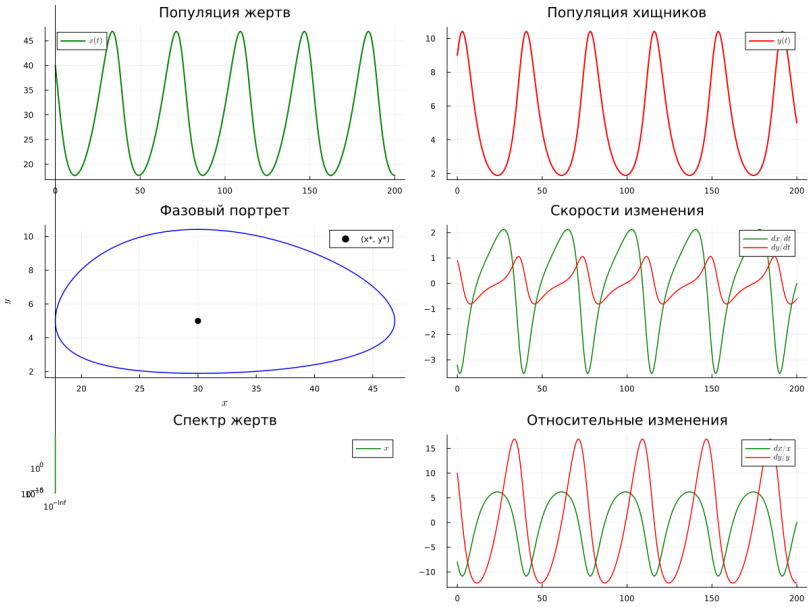


Рисунок 3.18: lv_panel

Результаты (рис. 3.19).

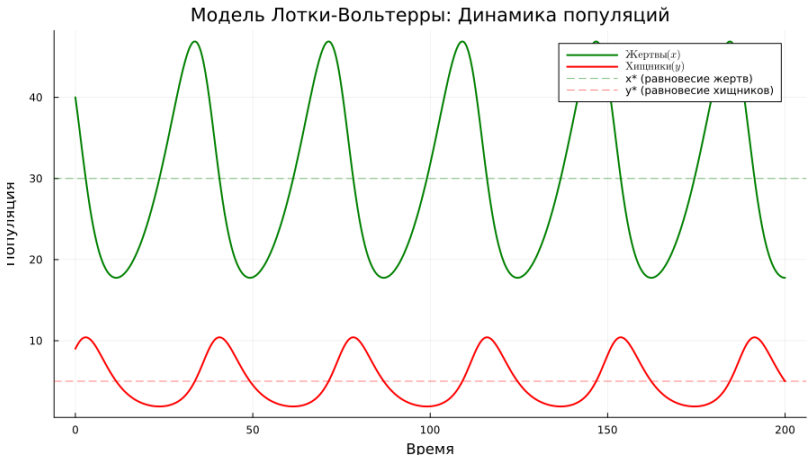


Рисунок 3.19: lv_dynamics

Результаты (рис. 3.20).

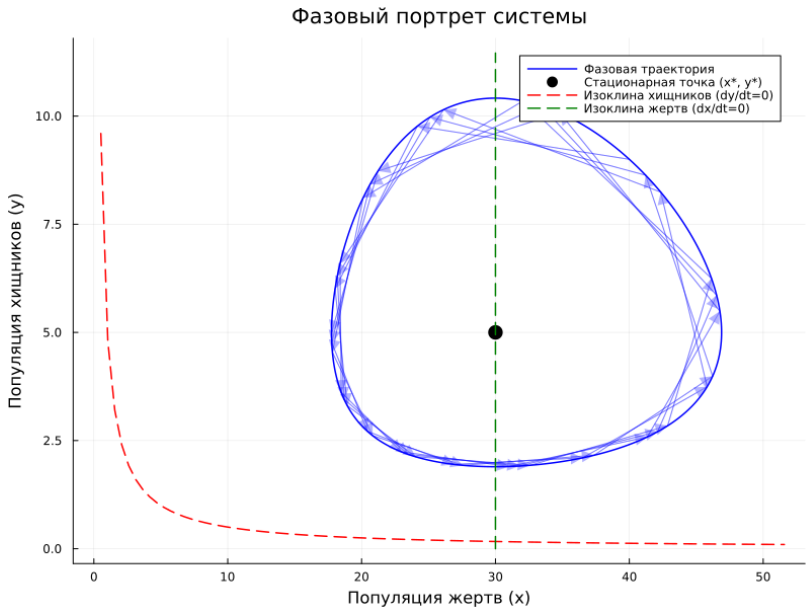


Рисунок 3.20: lv_phase_portrait

Результаты (рис. 3.21).

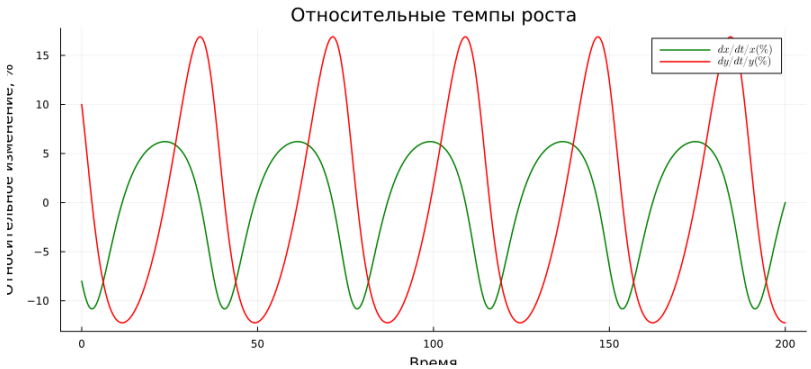


Рисунок 3.21: lv_relative_changes

Результаты (рис. 3.22).

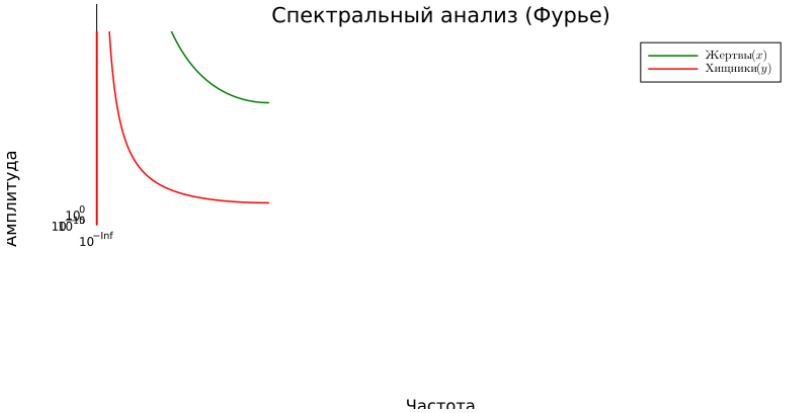


Рисунок 3.22: lv_spectrum

3.4 lv_params

```
# □□□□□□ □□□□□□□□□□□□□□□□ □ □□□□□□□□ □□□□□□□□□□
if false # □□□□□□□□□□ □ true □□□ □□□□□□□□□□ □□□□□□□
    println(" 1. □□□□□□□ □□□□□□□□ □□□□□□□□□□□ □□□□□ (□):")
    analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], "□")
    println(" 2. □□□□□□□ □□□□□□□□□□ □□□□□□□□ (□):")
    analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], "□")
end
```

```
println(" [redacted] [redacted] [redacted]!")

# # [redacted] [redacted] [redacted] [redacted] [redacted]
# [redacted] [redacted] [redacted] [redacted] [redacted] (alpha) [redacted] [redacted].

alphas = [0.5, 1.0, 1.5]
p_base = [1.0, 0.1, 0.1, 0.4]

for a in alphas
    p_new = [a, p_base[2], p_base[3], p_base[4]]
    prob_new = ODEProblem(lotka_volterra!, u0, tspan, p_new)
    sol_new = solve(prob_new)
    ## [redacted] [redacted] [redacted] [redacted] [redacted]
    display(plot(sol_new, title="[redacted] [redacted] alpha = $a"))
end
```

Показываю, Анализ модели при изменении параметров (рис. 3.23).

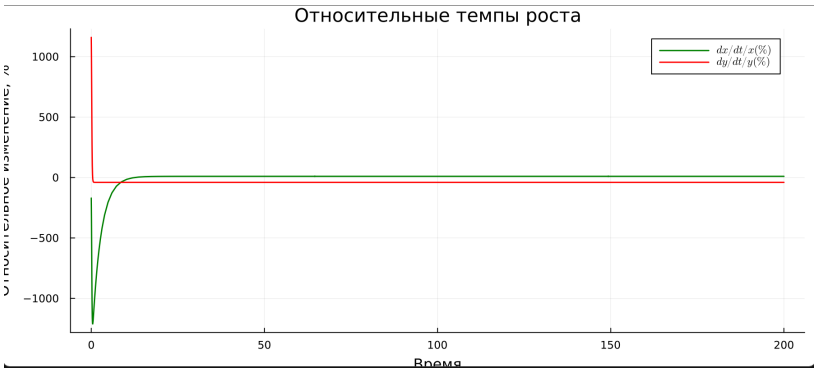


Рисунок 3.23: params_lv

Показываю, что есть все ноутбуки (рис. 3.24).

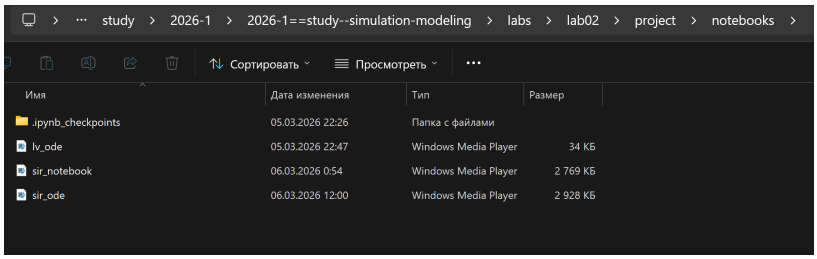


Рисунок 3.24: params_lv

Показываю, что есть все литературные файлы (рис. 3.25).

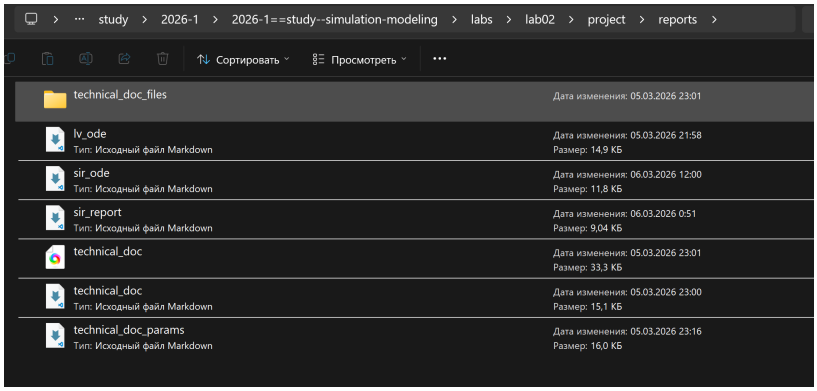


Рисунок 3.25: lit_code

4 Выводы

Построила имитационные модели на языке Julia Модель SIR и Модель Лотки–Вольтерры.

Список литературы